Rapport de performance

1. Contexte

Des lenteurs ont été observées sur la page **« Invités »** de l'application. Ce ralentissement devenait plus visible lorsque le nombre d'utilisateurs invités augmentait. Afin d'identifier l'origine du problème et d'évaluer les performances globales de l'application, une analyse de performance des pages du Front Office a été réalisée.

Les objectifs étaient les suivants :

- identifier la source du ralentissement
- mesurer les performances des pages du Front Office
- comparer les performances avant et après optimisation

2. Périmètre de l'analyse

Les pages suivantes du **Front Office** ont été analysées :

- Page d'accueil
- Page Portfolio
- Page Invités
- Page Invité (profil d'un invité)
- Page "Qui suis-je"

3. Outils utilisés

Le **Symfony Profiler** a été utilisé afin d'analyser les performances côté serveur.

Les indicateurs observés sont :

- temps total d'exécution de la requête
- temps d'exécution des requêtes SQL
- nombre de requêtes exécutées
- utilisation maximale de mémoire

Ces indicateurs permettent d'identifier les problèmes liés :

- aux requêtes base de données

- à l’exécution du code PHP
- à la consommation mémoire.

Lighthouse a été utilisé afin d’analyser les performances côté client.

Les indicateurs observés sont :

- score de performance global
- temps de chargement de la page

Ces mesures permettent d’évaluer :

- la rapidité d’affichage
- l’optimisation globale du chargement de la page.

4. Méthodologie

Chaque page a été testée selon la même procédure :

1. Mesure avant optimisation
2. Correction du problème identifié
3. Mesure après optimisation

Les résultats ont ensuite été comparés afin d’évaluer l’impact des optimisations.

5. Problème identifié : requêtes N+1

L’analyse réalisée avec **Symfony Profiler** a révélé que la lenteur de la page **Invités** était causée par un problème de **requêtes N+1**.

Dans ce cas :

- Doctrine exécute une première requête pour récupérer la liste principale des invités
- puis une requête supplémentaire pour chaque invité afin de récupérer ses données associées (par exemple les médias)

Ce comportement entraîne une augmentation importante du nombre de requêtes lorsque le nombre d’invités augmente.

Par exemple :

- 1 requête pour récupérer les invités

    - N requêtes pour récupérer leurs médias

Cela peut générer des dizaines de requêtes SQL, ce qui ralentit fortement le chargement de la page.

Cependant, l'analyse a également montré un second facteur de ralentissement : le **poids des images affichées**.

La page contient un grand nombre de photos, initialement au format **JPEG**, dont la taille est relativement importante. Or, le chargement simultané de nombreuses images lourdes a un impact direct sur :

- le temps d'affichage de la page ;
- le volume de données téléchargées ;
- les performances globales perçues par l'utilisateur.

Enfin, d'autres éléments contribuaient aussi au ralentissement global du rendu, notamment le chargement des ressources front-end, comme le fichier CSS principal lorsqu'il n'est pas optimisé.

6. Solution

Pour résoudre ces problèmes, plusieurs optimisations complémentaires ont été mises en œuvre.

Optimisation des requêtes en base des données

Une méthode spécifique a été créée dans le repository.
La requête a été réécrite afin d'inclure directement les jointures nécessaires avec la table **media**.

Cela permet de :

- récupérer toutes les données nécessaires ;
- en **une seule requête optimisée** ;
- éviter l'exécution de requêtes supplémentaires pour chaque invité.

Cette approche supprime le problème de **requêtes N+1** et réduit nettement la charge côté serveur.

Mise en cache

Un système de caching a également été ajouté afin d'améliorer le temps de réponse sur les pages fréquemment consultées.

Pagination

La pagination a été mise en place pour limiter le nombre d'éléments affichés à la fois, et donc réduire le nombre d'images volumineuses chargées simultanément.

Optimisation des images

Les images ont été optimisées afin de réduire leur poids et d'améliorer leur chargement sur le site. Les fichiers JPEG, souvent trop lourds pour une utilisation web, ont été compressés et convertis dans un format plus adapté.

L'optimisation a été réalisée avec **LiipImagineBundle**, permettant notamment :

- la compression des images ;
- la génération d'images optimisées pour le web ;
- l'utilisation du format **WebP**, plus léger que le JPEG.

Optimisation du chargement des images

Le chargement des images a aussi été amélioré côté front-end grâce aux attributs HTML suivants :

loading="{{ isFirst ? 'eager' : 'lazy' }}"

fetchpriority="{{ isFirst ? 'high' : 'auto' }}"

decoding="async"

Ces attributs permettent de :

- charger en priorité l'image la plus importante visible immédiatement à l'écran ; différer le chargement des autres images avec le **lazy loading** ;
- améliorer le rendu perçu grâce au décodage asynchrone des images.

Cette stratégie réduit le temps de chargement initial et améliore les performances globales de la page.

Optimisation des ressources CSS

Le fichier CSS global a été **modifié** afin de réduire sa taille.La minification a été réalisée avec **SensioLabs Minify Bundle**. Elle consiste à supprimer les espaces, retours à la ligne et caractères inutiles dans le fichier, ce qui permet :

- un téléchargement plus rapide ;
- un rendu plus efficace ;
- une amélioration globale du temps de chargement du site.

7. Résultats

Après l'optimisation :

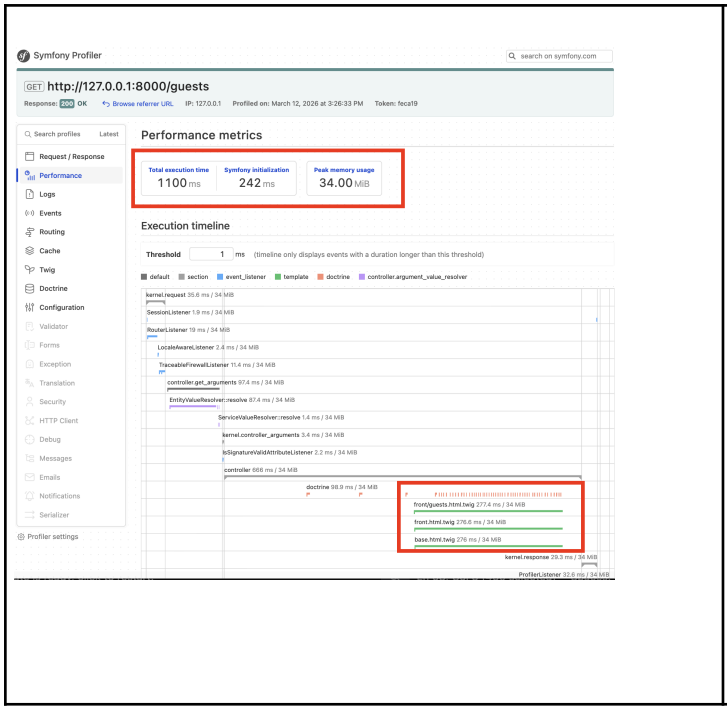
- le nombre de requêtes SQL a diminué
- le temps d'exécution total a été réduit
- le score de performance Lighthouse a été amélioré

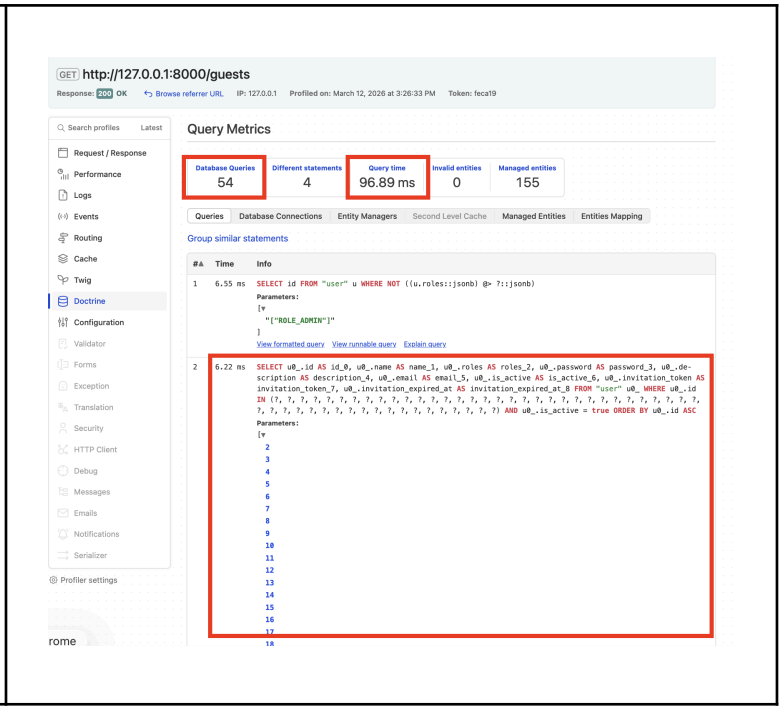
8. Analyse des pages

Page invités

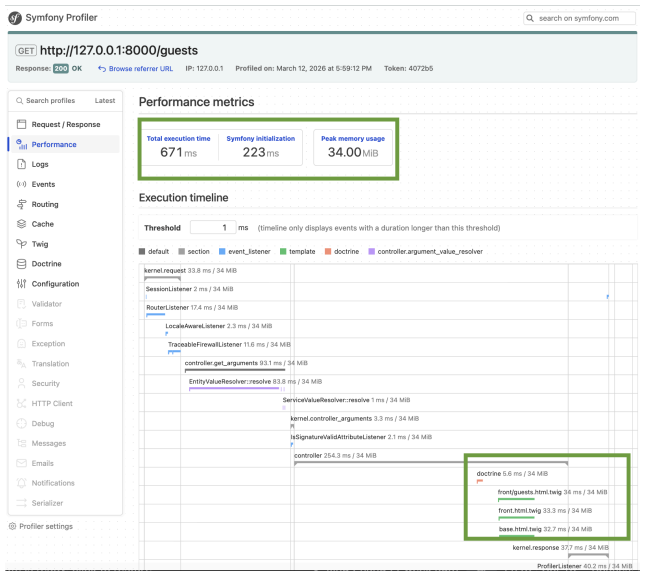
	Avant	Après	Evolution
Temps total d'exécution(ms)	1100	671	- 39%
Nombre de requêtes vers la BD	54	1	- 98%
Temps d'exécution des requêtes(ms)	96.89	5,57	- 94%
Score de performance	85	91	+ 7%
FCP(s)	3,3s	2.5s	- 24%
LCP(s)	3,4s	2.7s	- 21%

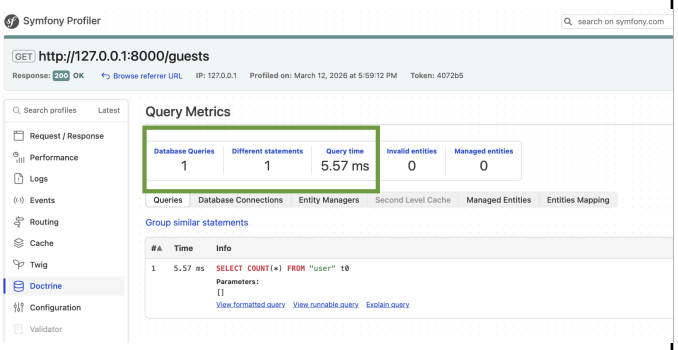
Symfony profiler (Avant Optimisation)



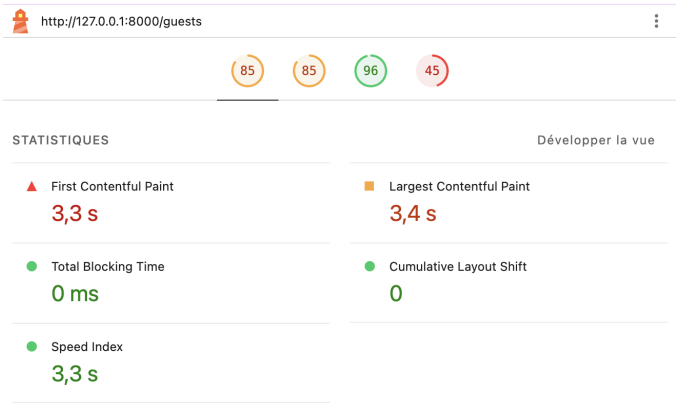
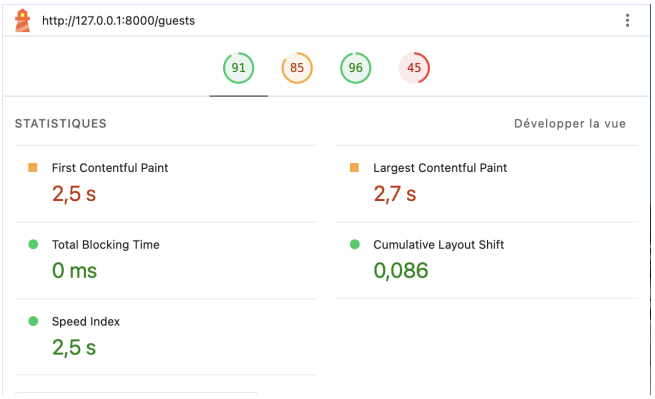


Symfony profiler (Après optimisation)





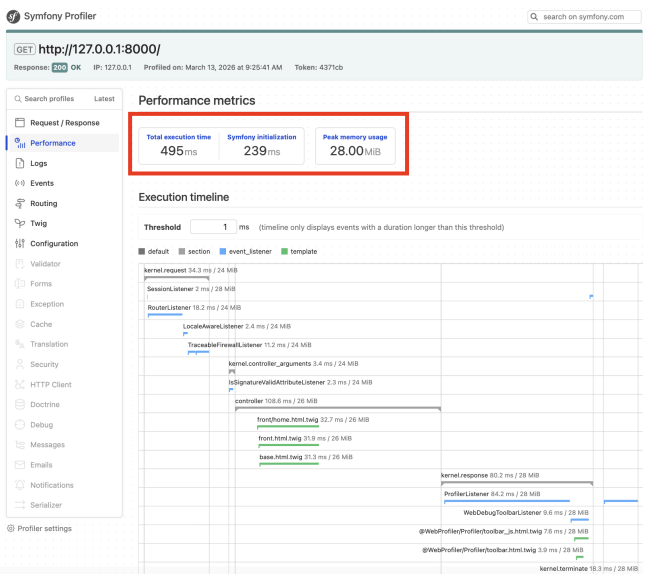
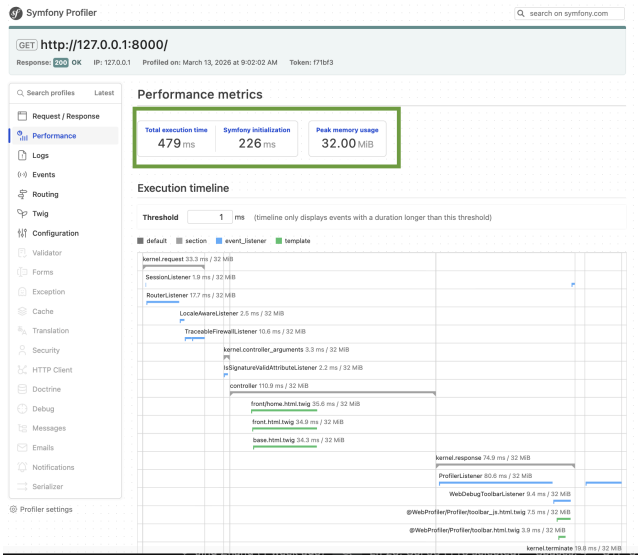
Lighthouse

(Avant Optimisation)	(Après optimisation)
	

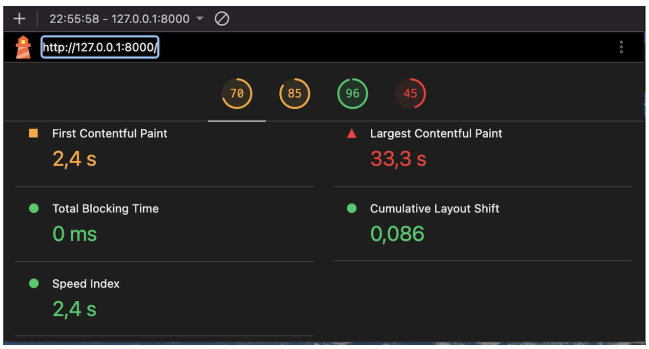
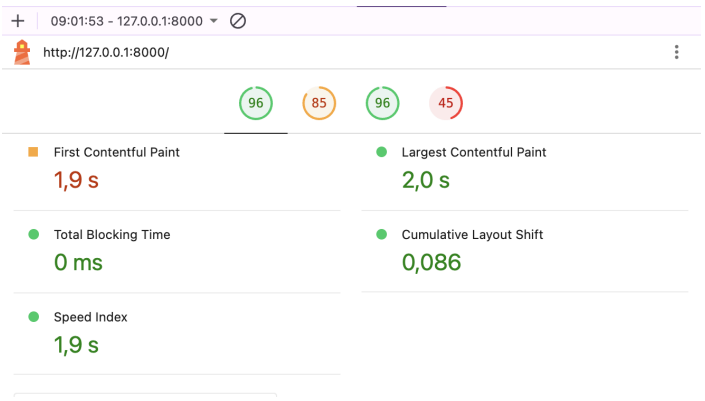
Page d’accueil

	Avant	Après	Evolution
Temps total d’exécution(ms)	495	479	- 3%
Score de performance	70	96	+ 37%
FCP(s)	2,4s	1,9s	- 21%
LCP(s)	33,3s	2.0s	- 94%

Symfony Profiler

(Avant Optimisation)	(Après optimisation)
	

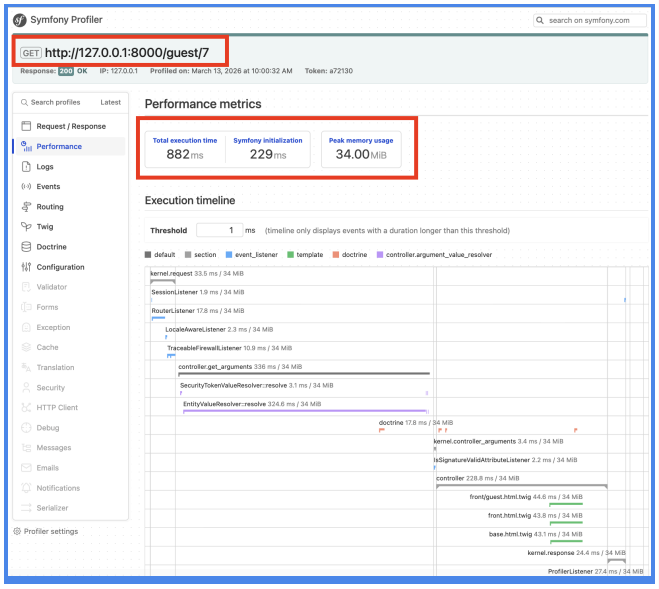
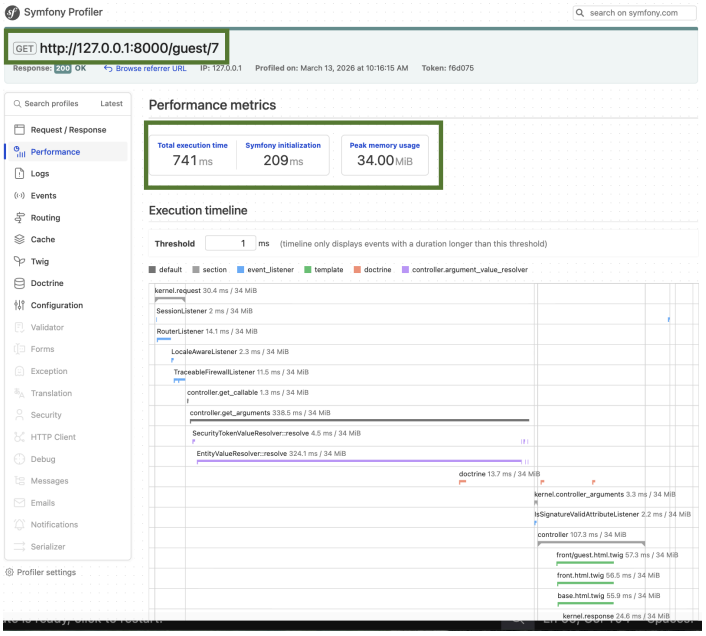
Lighthouse

(Avant Optimisation)	(Après optimisation)
	

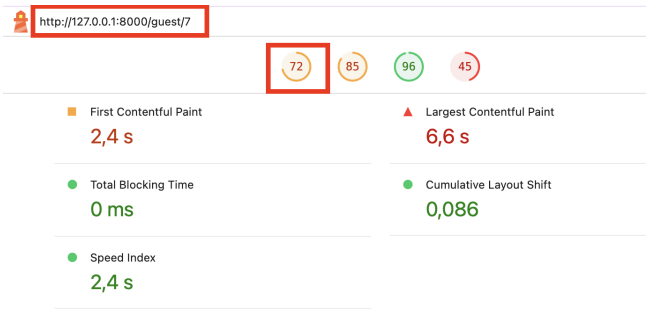
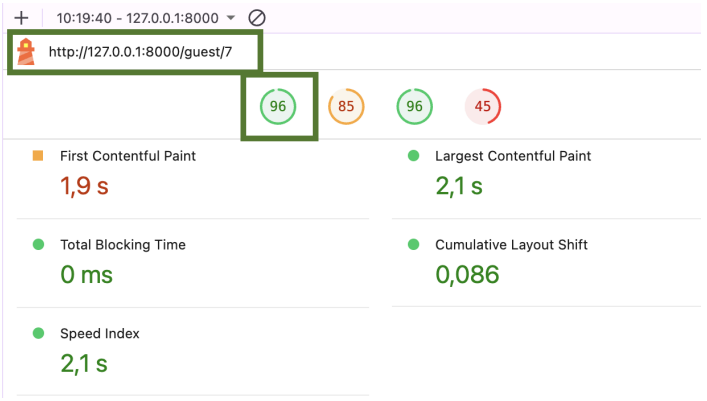
Page invité

	Avant	Après	Evolution
Temps total d’exécution(ms)	882	741	- 3%
Score de performance	72	96	+ 33.3%
FCP(s)	2,4s	1,9s	- 21%
LCP(s)	6.6s	2,1s	- 68.2%

Symfony Profiler

(Avant Optimisation)	(Après optimisation)
	

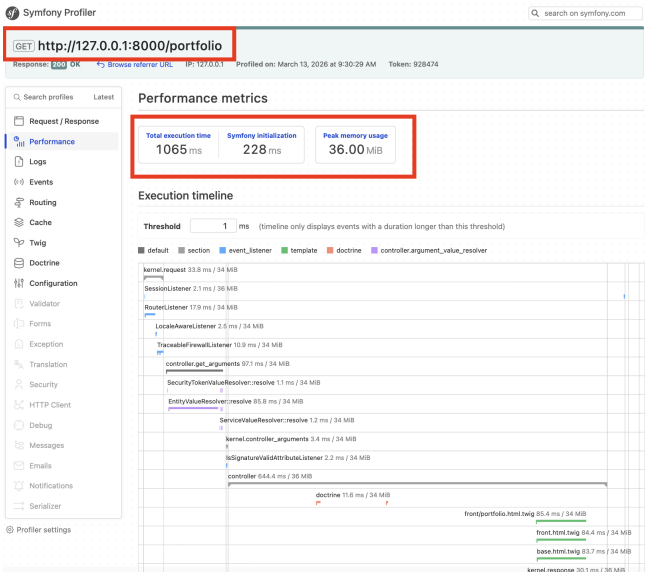
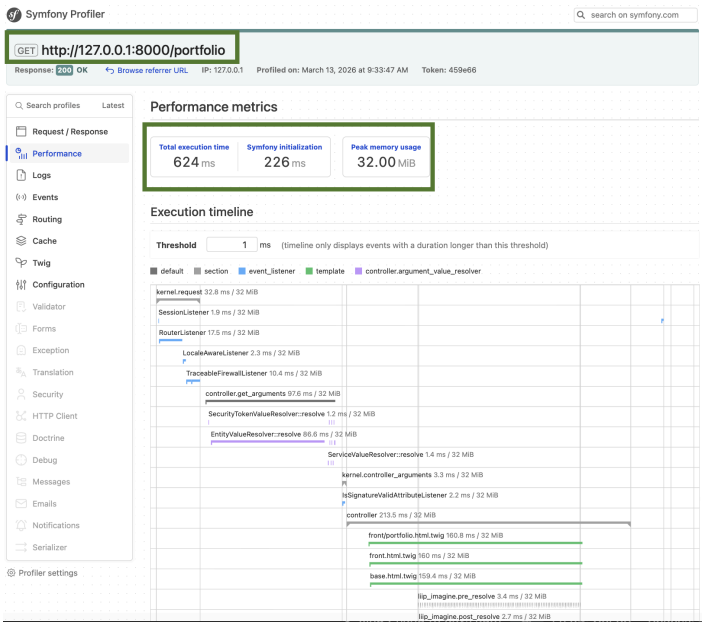
Lighthouse

(Avant Optimisation)	(Après optimisation)
	

Page porfolio

	Avant	Après	Evolution
Temps total d’exécution(ms)	1065	624	- 41.4%
Score de performance	71	91	+ 28%
FCP(s)	2,6s	2,2s	- 15.3%
LCP(s)	29.8s	3.2s	- 89,2%

Symfony Profiler

(Avant Optimisation)	(Après optimisation)
	

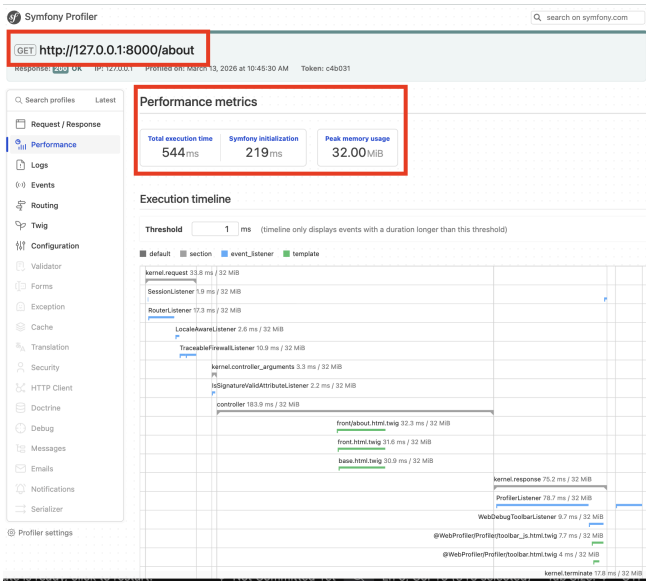
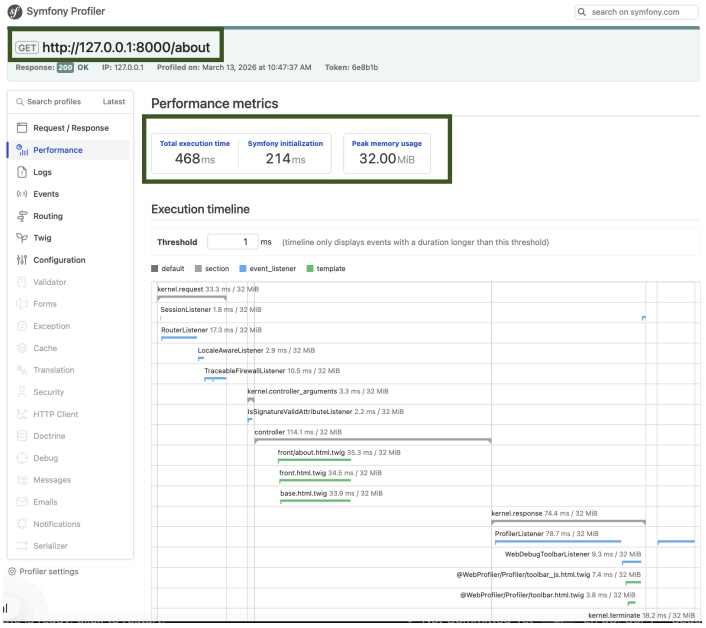
Lighthouse

(Avant Optimisation)	(Après optimisation)
http://127.0.0.1:8000/portfolio 71 85 96 45 First Contentful Paint 2,6 s Largest Contentful Paint 29,8 s Total Blocking Time 0 ms Cumulative Layout Shift 0 Speed Index 2,7 s	http://127.0.0.1:8000/portfolio 91 85 96 45 STATISTIQUES Développer la vue First Contentful Paint 2,2 s Largest Contentful Paint 3,2 s Total Blocking Time 0 ms Cumulative Layout Shift 0 Speed Index 2,2 s

Page “qui suis-je”

	Avant	Après	Evolution
Temps total d’exécution(ms)	544	468	-13,9%
Score de performance	84	95	+13,1%
FCP(s)	2,7	2,1	- 28,5%
LCP(s)	3,6	2,1	- 41,7%

Symfony Profiler

(Avant Optimisation)	(Après optimisation)
	

Lighthouse

(Avant Optimisation)	(Après optimisation)
http://127.0.0.1:8000/about 84 78 96 36 First Contentful Paint 2,7 s Largest Contentful Paint 3,6 s Total Blocking Time 0 ms Cumulative Layout Shift 0,086 Speed Index 2,7 s	http://127.0.0.1:8000/about 95 85 96 45 First Contentful Paint 2,1 s Largest Contentful Paint 2,1 s Total Blocking Time 0 ms Cumulative Layout Shift 0,086 Speed Index 2,1 s